

fpos Requirements

- [Introduction](#)
- [Revision History](#)
- [Discussion](#)
- [Resolved Issues](#)
- [Proposed Resolution](#)
- [Bibliography](#)
- [Acknowledgements](#)

Introduction

This is a plain wording fixing proposal for the table-based requirement set of class template fpos. A paper is provided instead of drafting wording in the referenced issues, to have more room available for presentation and explanation.

The wording changes suggested by this paper *should not* have effects on any existing implementations, because it mainly intended to clarify several underspecified parts of the current specification. Nonetheless during the analysis of the paper there occurred some curiosities in the specification that caused to ask some questions to the committee which *might* lead to decisions to change existing implementations, but the author has tried to be as conservative as possible in regard to such suggestions.

Revision History

Changes since [P0759R0](#):

- Rebases to the reference working draft [N4750](#)
- Get rid of the expression $q + ol == p$ in Table 106 — "Position type requirements" including the additionally needed expression symbol `ol` that was needed only to specify the expression $p - q$. The author believes that the newly added assertion expression $p == q + (p - q)$ is sufficient.
- Add an operational semantics entry $p + o$ to the expression row $o + p$ in Table 106.

Discussion

This paper revision does not repeat the sections [Discussion](#), [1. Specification Facts](#), and [2. Specification Problems](#) of its predecessor [P0759R0](#).

Please refer to the previous paper regarding general background and (extended) rationale.

Resolved Issues

If the proposed resolution will be accepted, the following library issues will be resolved:

Number	Description	C++17 NB comment
2808	Requirements for <code>fpos</code> and <code>stateT</code>	GB 60
2832	§[fpos.operations] strange requirement for <code>P(i)</code>	—

Proposed resolution

At some places below, additional markup of the form

[Drafting notes: whatever — end drafting notes]

is provided, which is *not* part of the normative wording, but is solely shown to provide additional information to the reader about the rationale of the concrete wording.

The proposed wording changes refer in all cases to [N4750](#).

1. Change 30.5.4.2 [fpos.operations] as indicated:

~~-1- Operations specified in Table 112 are permitted.~~ An `fpos` type specifies file position information. It holds a state object whose type is equal to the template parameter `stateT`. Type `stateT` shall meet the `DefaultConstructible` (Table 22), `CopyConstructible` (Table 24), `CopyAssignable` (Table 26), and `Destructible` (Table 27) requirements. If `is_trivially_copy_constructible_v<stateT>` is true, then `fpos<stateT>` has a trivial copy constructor. If `is_trivially_copy_assignable<stateT>` is true, then `fpos<stateT>` has a trivial copy assignment operator. If `is_trivially_destructible_v<stateT>` is true, then `fpos<stateT>` has a trivial destructor. All specializations of `fpos` satisfy the `DefaultConstructible`, `CopyConstructible`, `CopyAssignable`, `Destructible`, and `EqualityComparable` (Table 20) requirements. In addition, the expressions shown in Table 112 are valid and have the indicated semantics. In that table,

(1.1) — `P` refers to an instance of `fpos`,

(1.2) — `p` and `q` refer to values of type `P` or `const P`,

(1.?) — `p1` and `q1` refer to modifiable lvalues of type `P`,

(1.3) — `o` refers to type `streamoff`, and

(1.4) — `o` refers to a value of type `streamoff` or `const streamoff`

~~(1.5) — `sz` refers to a value of type `streamsize` and~~

~~(1.6) — `i` refers to a value of type `int`.~~

[Drafting notes: It is recommend to strike the non-normative note in p2 completely. This seems to be very out-dated wording and what is says is already said in 20.4.1.3 [structure.requirements], in particular in p3+p4 — end drafting notes]

~~-2- [Note: Every implementation is required to supply overloaded operators on fpos objects to satisfy the requirements of 30.5.4.2 [fpos.operations]. It is unspecified whether these operators are members of fpos, global operators, or provided in some other way. — end note]~~

-3- Stream operations that return a value of type traits::pos_type return P(0(-1)) as an invalid value to signal an error. If this value is used as an argument to any istream, ostream, or streambuf member that accepts a value of type traits::pos_type then the behavior of that function is undefined.

2. Change Table 106 — "Position type requirements", as indicated:

[Drafting notes: We can strike the wording about the destructor and ==, because we have now the EqualityComparable and Destructible requirements specified in 30.5.4.2 [fpos.operations] — end drafting notes]

Table 106 — Position type requirements

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
P(i)	—	—	p == P(i) note: a destructor is assumed.
P p(i); P p = i;	—	—	Postconditions: p == P(i);
P(o)	fpos P	converts from offset	<u>Effects: Value-initializes the state object.</u>
<u>P p(o);</u> <u>P p = o;</u>			<u>Effects: Value-initializes the state object.</u> <u>Postconditions: p == P(o).</u>
<u>P().</u>	<u>P</u>	<u>P(0).</u>	
<u>P p;</u>		<u>P p(0);</u>	
O(p)	streamoff	converts to offset	P(O(p)) == p
p == q	convertible to bool	—	== is an equivalence relation
p != q	convertible to bool	!(p == q)	
q = p + o p += o	fpos P	+ offset	<u>Remarks: With ql = p + o;, then:</u> <u>ql - o == p</u>
<u>pl += o</u>	<u>P&</u>	<u>+= offset</u>	<u>Remarks: With ql = pl; before the +=, then:</u> <u>pl - o == ql</u>
q = p - o p -= o	fpos P	- offset	<u>Remarks: With ql = p - o;, then:</u> <u>ql + o == p</u>
<u>pl -= o</u>	<u>P&</u>	<u>-= offset</u>	<u>Remarks: With ql = pl; before the -=, then:</u> <u>pl + o == ql</u>
<u>o + p</u>	<u>convertible to P</u>	<u>p + o</u>	<u>P(o + p) == p + o</u>
o = p - q	streamoff	distance	<u>p == q + (p - q).</u> q + o == p

streamsize(o)	streamsize	converts	streamsize(0(sz)) == sz
0(sz)	streamoff	converts	streamsize(0(sz)) == sz

Bibliography

[N4750 Richard Smith: "Working Draft, Standard for Programming Language C++"](#)

Acknowledgements

Thanks to Jonathan Wakely, Billy Robert O'Neal III, and Axel Naumann for a review of this proposal and for encouragement to make a little more changes as I originally believed I should make.